

SENAC RJ — Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

SPRINT 4 — APLICAÇÃO PRÁTICA

Tema: DDD — Entendendo o Domínio do Problema

Entrega: 29/05/2026

Este documento apresenta a **modelagem táctica de domínio** do sistema fictício de **Gestão de Entregas Logísticas**, aplicando os blocos de construção do DDD: Entidades, Value Objects, Agregados e Repositórios. O objetivo é demonstrar como os conceitos da Sprint 3 (Domínio, Linguagem Ubíqua e Bounded Context) se traduzem em estruturas concretas de código e modelagem.

1. Visão Geral do Sistema Fictício

O sistema de **Gestão de Entregas Logísticas** é uma plataforma B2B que permite a empresas (remetentes) contratar o transporte de mercadorias até os seus clientes (destinatários). O sistema gerencia todo o ciclo: criação do pedido, roteirização automática, execução da entrega por motoristas e rastreamento em tempo real.

1.1 Bounded Contexts em Escopo para Esta Sprint

Bounded Context	Foco desta Sprint
Pedidos	Modelagem completa: Pedido (agregado raiz), Item, Endereço, Remetente.
Roteirização	Modelagem completa: Rota (agregado raiz), Parada, Veículo, Motorista.
Rastreamento	Modelagem parcial: Remessa e EventoDeRastreamento.

2. Blocos de Construção do DDD Tático

Entidade	Objeto com identidade única e persistente que evolui ao longo do tempo. Dois objetos com os mesmos atributos mas IDs diferentes são entidades distintas. Ex.: Pedido com ID #1042, mesmo que seus itens mudem.
Value Object	Objeto imutável definido pelos seus atributos, sem identidade própria. Dois value objects com os mesmos atributos são iguais. Ex.: Endereço — 'Rua A, 100, SP' é igual a qualquer outro objeto com os mesmos valores.
Agregado	Cluster de entidades e value objects tratados como uma única unidade de consistência. Toda operação de escrita passa pela Raiz do Agregado (Aggregate Root), que garante as invariantes de negócio.
Repositório	Abstração que encapsula a lógica de persistência de um Agregado. Oferece uma interface semelhante a uma coleção em memória, ocultando detalhes de banco de dados. Existe um repositório por Agregado.
Serviço de Domínio	Operação de domínio que não pertence naturalmente a nenhuma entidade ou value object. Encapsula lógica de negócio que envolve múltiplos agregados. Ex.: CalculadoraDeTarifa, RoteirizadorDeEntregas.
Evento de Domínio	Fato relevante que ocorreu no domínio, expressado no passado. Usado para comunicação entre bounded contexts e para gatilhos de reação. Ex.: PedidoCriado, EntregaRealizada, RotaCalculada.

3. Bounded Context: Pedidos

3.1 Entidades e Value Objects

Nome	Tipo	Atributos Principais	Invariantes / Regras de Negócio
Pedido	Entidade (Raiz do Agregado)	id, status, dataCriacao, itens, enderecoEntrega, idRemetente, valorTotal	Deve ter ao menos 1 Item. Status só pode avançar (CRIADO→COLETADO→EM_ROTA→ENTREGUE). ValorTotal recalculado ao adicionar/remover itens.
Item	Entidade (dentro do Agregado Pedido)	id, descricao, pesoKg, altura, largura, comprimento, tipo (FRAGIL, REFRIGERADO, NORMAL)	Peso deve ser > 0. Dimensões devem ser positivas. Tipo não pode ser alterado após coleta.
Endereco	Value Object	logradouro, numero, complemento, bairro, cidade, estado, cep	CEP deve ter formato válido (8 dígitos). Cidade e Estado não podem ser vazios. Imutável após criação.
Remetente	Entidade (referenciada, não no Agregado)	id, razaoSocial, cnpj, email, telefone	CNPJ deve ser único e válido. Email deve ter formato válido.

3.2 Estrutura do Agregado Pedido

O **Pedido** é a raiz do agregado. Toda operação que modifica um Item ou o Endereço de Entrega deve passar pela raiz, garantindo as invariantes. O ID do Pedido é a única referência externa ao agregado.

AGREGADO: Pedido
• Raiz: Pedido (PedidoId, status, dataCriacao, valorTotal) • Entidade interna: Item (ItemId, descricao, peso, dimensoes, tipo) • Value Object: Endereco (logradouro, numero, bairro, cidade, estado, cep) • Value Object: StatusPedido (enum: CRIADO COLETADO EM_ROTA ENTREGUE CANCELADO) • Referência externa: RemetenteId (apenas o ID — não carrega o objeto Remetente) • Evento emitido: PedidoCriado, ItemAdicionado, PedidoCancelado

3.3 Repositório de Pedidos

O **PedidoRepository** é a única porta de acesso à persistência do agregado Pedido. A interface abaixo é definida na camada de domínio; a implementação concreta fica na camada de infraestrutura (ex.: Prisma, JPA).

```
// Interface definida no domínio
interface PedidoRepository {
  save(pedido: Pedido): Promise<void>;
  findById(id: PedidoId): Promise<Pedido | null>;
  findByRemetente(remetenteId: RemetenteId): Promise<Pedido[]>;
  findByStatus(status: StatusPedido): Promise<Pedido[]>;
  delete(id: PedidoId): Promise<void>;
}
```

4. Bounded Context: Roteirização

4.1 Entidades e Value Objects

Nome	Tipo	Atributos Principais	Invariantes / Regras de Negócio
Rota	Entidade (Raiz do Agregado)	id, data, paradas, idVeiculo, idMotorista, status, distanciaTotalKm	Deve ter ao menos 1 Parada. Distância total calculada automaticamente. Status: PLANEJADA→EM_EXECUCAO→CONCLUIDA.
Parada	Entidade (dentro do Agregado Rota)	id, ordemNaRota, idPedido, janelaTempo, situacao, coordenadasGPS	Ordem deve ser única na Rota. Janela de tempo não pode ser retroativa. Situacao: PENDENTE REALIZADA FALHA.
JanelaTempo	Value Object	horaInicio, horaFim	horaFim deve ser posterior a horaInicio. Intervalos de no mínimo 30 minutos. Imutável.
Coordenadas	Value Object	latitude, longitude	Latitude: -90 a 90. Longitude: -180 a 180. Imutável.
Veiculo	Entidade (referenciada)	id, placa, tipo, capacidadeKg, capacidadeM3, ativo	Placa deve ser única. Capacidade > 0. Não pode ser alocado a duas rotas simultâneas.
Motorista	Entidade (referenciada)	id, nome, cnh, validadeCnh, ativo	CNH deve ser válida e vigente. Não pode dirigir mais de 8h/dia. Não pode ser alocado a duas rotas simultâneas.

4.2 Estrutura do Agregado Rota

AGREGADO: Rota

• **Raiz:** Rota (RotaId, data, distanciaTotalKm, status) • **Entidade interna:** Parada (ParadaId, ordemNaRota, situacao, janelaTempo, coordenadas) • **Value Object:** JanelaTempo (horaInicio, horaFim) • **Value Object:** Coordenadas (latitude, longitude) • **Value Object:** StatusRota (enum: PLANEJADA | EM_EXECUCAO | CONCLUIDA | CANCELADA) • **Referências externas:** VeiculoId, MotoristaId, PedidoId (por Parada) • **Eventos emitidos:** RotaCalculada, ParadaRealizada, RotaConcluida

4.3 Serviço de Domínio — RoteirizadorDeEntregas

A lógica de otimização de rotas não pertence a nenhuma entidade isolada — ela opera sobre múltiplos Pedidos e Veículos. Por isso é encapsulada em um **Serviço de Domínio**.

```
// Serviço de Domínio — pertence à camada de domínio
interface RoteirizadorDeEntregas {
  calcularRota(
    pedidos: Pedido[],
    veiculo: Veiculo,
    motorista: Motorista,
    dataExecucao: Date
  ): Rota;
}

// Regras de negócio aplicadas internamente:
// 1. Peso total dos pedidos <= capacidadeKg do veículo
// 2. Volume total <= capacidadeM3 do veículo
// 3. Janelas de tempo dos pedidos devem ser respeitadas
// 4. Motorista não pode exceder 8h de jornada
```

4.4 Repositórios da Roteirização

```
interface RotaRepository {
  save(rota: Rota): Promise<void>;
  findById(id: RotaId): Promise<Rota | null>;
  findByMotoristaAndData(id: MotoristaId, data: Date): Promise<Rota[]>;
  findByVeiculoAndData(id: VeiculoId, data: Date): Promise<Rota[]>;
}

interface VeiculoRepository {
  findDisponiveis(data: Date, capacidadeMinKg: number): Promise<Veiculo[]>;
  findById(id: VeiculoId): Promise<Veiculo | null>;
}
```

5. Bounded Context: Rastreamento

O contexto de Rastreamento é um **bounded context downstream** que consome eventos dos contextos de Pedidos e Roteirização para construir sua visão do estado de uma Remessa. Aplica o padrão

Anti-Corruption Layer para não depender diretamente do modelo de Pedido.

5.1 Entidades e Value Objects

Nome	Tipo	Atributos Principais	Invariantes / Regras de Negócio
Remessa	Entidade (Raiz do Agregado)	id, codigoRastreio, idPedido, status, etaAtual, eventos, ultimaLocalizacao	Código de rastreio único e imutável. ETA recalculado a cada novo EventoDeRastreamento. Status avança de forma unidirecional.
EventoDeRastreamento	Value Object (imutável)	tipo, descricao, dataHora, localizacao (Coordenadas)	DataHora não pode ser futura. Tipo: COLETADO SAIU_PARA_ENTREGA ENTREGUE TENTATIVA_FALHA DEVOLVIDO. Imutável após registro.
ETA	Value Object	dataHoraEstimada, confiancaPercentual	Confiança: 0 a 100%. Atualizado pelo serviço de cálculo de ETA. Imutável — nova instância a cada atualização.

5.2 Repositório e Integração

```
interface RemessaRepository {
  save(remessa: Remessa): Promise<void>;
  findByCodigoRastreio(codigo: string): Promise<Remessa | null>;
  findByPedidoId(id: PedidoId): Promise<Remessa | null>;
}

// Integração via eventos de domínio (Anti-Corruption Layer)
// Quando Pedido emite PedidoCriado →
// Handler cria uma Remessa com status AGUARDANDO_COLETA
// Quando Rota emite ParadaRealizada →
// Handler adiciona EventoDeRastreamento à Remessa correspondente
```

6. Eventos de Domínio — Fluxo de Integração

Os **Eventos de Domínio** são o mecanismo de comunicação entre bounded contexts, garantindo baixo acoplamento. O diagrama abaixo mostra o fluxo principal do sistema:

Passo	Contexto	Ação / Evento
1. Remetente cria um Pedido	Pedidos	→ emite: PedidoCriado
2. PedidoCriado recebido	Roteirização	→ agenda Rota; emite: RotaCalculada
3. PedidoCriado recebido	Rastreamento	→ cria Remessa (AGUARDANDO_COLETA)
4. Motorista inicia a Rota	Roteirização	→ emite: RotaIniciada

5. RotaIniciada recebido	Rastreamento	→ atualiza Remessa (EM_ROTA); recalcula ETA
6. Parada realizada	Roteirização	→ emite: ParadaRealizada (com PedidoId)
7. ParadaRealizada recebido	Rastreamento	→ adiciona EventoDeRastreamento (ENTREGUE)
8. Última parada da Rota realizada	Roteirização	→ emite: RotaConcluida
9. RotaConcluida recebido	Pedidos	→ atualiza status Pedido para ENTREGUE

7. Tabela Consolidada da Modelagem

A tabela a seguir consolida todos os artefatos identificados na modelagem tática:

Contexto	Artefato	Tipo	Papel no DDD
Pedidos	Pedido	Entidade / Raiz	Ponto de entrada do agregado; garante invariantes.
Pedidos	Item	Entidade	Unidade de carga gerenciada dentro do agregado.
Pedidos	Endereco	Value Object	Imutável, comparado por valor.
Pedidos	StatusPedido	Value Object (enum)	Representa o estado do ciclo de vida do Pedido.
Pedidos	PedidoRepository	Repositório	Abstrai persistência do agregado Pedido.
Pedidos	Remetente	Entidade (ref.)	Referenciada pelo ID; gerenciada fora do agregado.
Roteirização	Rota	Entidade / Raiz	Agrega as Paradas; garante consistência da sequência.
Roteirização	Parada	Entidade	Ponto de entrega com ordem e situação.
Roteirização	JanelaTempo	Value Object	Intervalo imutável de horário para uma Parada.
Roteirização	Coordenadas	Value Object	Par lat/long imutável.
Roteirização	Veiculo	Entidade (ref.)	Referenciada pelo ID na Rota.
Roteirização	Motorista	Entidade (ref.)	Referenciada pelo ID na Rota.
Roteirização	RoteirizadorDeEntregas	Serviço de Domínio	Calcula a Rota ótima para um conjunto de Pedidos.

Roteirização	RotaRepository	Repositório	Abstrai persistência do agregado Rota.
Rastreamento	Remessa	Entidade / Raiz	Agrega os eventos; representa o estado de rastreamento.
Rastreamento	EventoDeRastreamento	Value Object	Fato imutável ocorrido na vida da Remessa.
Rastreamento	ETA	Value Object	Estimativa imutável calculada a cada atualização.
Rastreamento	RemessaRepository	Repositório	Abstrai persistência do agregado Remessa.

8. Considerações Finais

Esta sprint demonstrou como os conceitos estratégicos do DDD (Domínio, Linguagem Ubíqua e Bounded Contexts, apresentados na Sprint 3) se materializam em artefatos táticos concretos: entidades com identidade e ciclo de vida, value objects imutáveis definidos por valor, agregados que garantem consistência e repositórios que abstraem a persistência.

A escolha de quais objetos são entidades (e portanto precisam de um ID) versus value objects (definidos por seus atributos) não é arbitrária — ela reflete diretamente as regras de negócio. Por exemplo, **Endereço** é um value object no contexto de Pedidos porque dois pedidos com o mesmo endereço de entrega são tratados de forma idêntica pela lógica de negócio. Já **Parada** é uma entidade porque ela tem um ciclo de vida próprio (PENDENTE → REALIZADA / FALHA) que precisa ser rastreado individualmente.

O sistema de entregas logísticas possui uma arquitetura orientada a eventos que desacopla os bounded contexts, permitindo que cada um evolua de forma independente. Esta é uma das principais vantagens do DDD em sistemas de grande escala.